计算机学院 操作系统 课程实验报告

实验题目： 实验 2 线程和管道控制		学号：202200130077
日期：2024.3.19	班级：智能班	姓名：薛润泽
Email：		
实验方法介绍： 使用虚拟机环境下的 Ubuntu22.04 环境，编写 c 语言程序，并配合在终端中运行，实现多线程的创建以及多线程之间的信息传递。		
实验过程描述： **一、示例实验描述：** (1) 建立 tpipe.c 的 C 语言程序，建立程序的 Makefile 文件		

tpipe.c

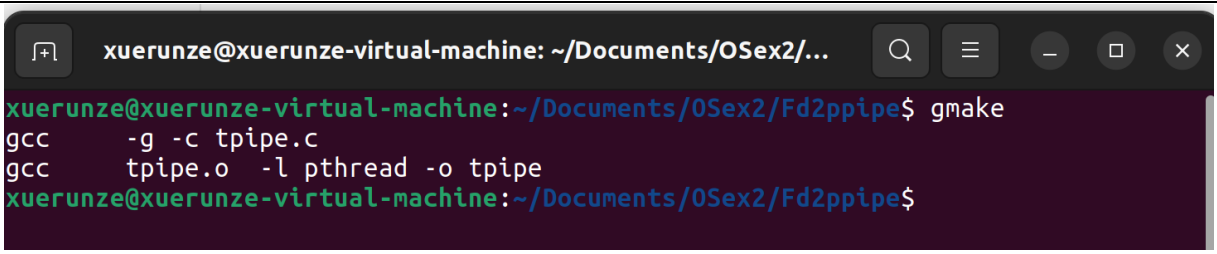
home > xuerunze > Documents > OSex2 > Fdppipe > tpipe.c

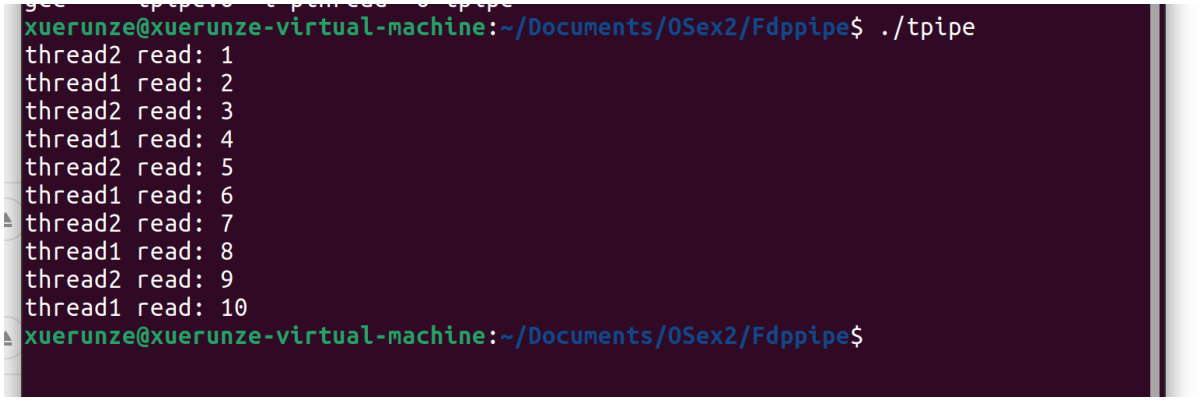
```c
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>
void task1(int *); //线程 1 执行函数原型
void task2(int *); //线程 2 执行函数原型
int pipe1[2],pipe2[2]; //存放第两个无名管道标号
pthread_t thrd1,thrd2; //存放第两个线程标识
int main(int argc,char *arg[])
{
int ret;
int num1,num2;
//使用 pipe()系统调用建立两个无名管道。建立不成功程序退出，执行终止
if(pipe(pipe1) < 0){
perror("pipe1 not create");
exit(EXIT_FAILURE);
}
if(pipe(pipe2) < 0)
{
perror("pipe2 not create");
exit(EXIT_FAILURE);
}
//使用 pthread_create 系统调用建立两个线程。建立不成功程序退出，执行终止
num1 = 1 ;
ret = pthread_create(&thrd1,NULL,(void *) task1,(void *) &num1);
if(ret){
perror("pthread_create: task1");
exit(EXIT_FAILURE);
}
num2 = 2 ;
ret = pthread_create(&thrd2,NULL,(void *) task2,(void *) &num2);
if(ret){
perror("pthread_create: task2");
exit(EXIT_FAILURE);
}
//挂起当前线程，等待线程 thrd2 结束，并回收其资源
pthread_join(thrd2,NULL);
//挂起当前线程，等待线程 thrd1 结束，并回收其资源
```

Open | Makefile ~/Documents/OSex2/Fdppipe | Save

```
src = tpipe.c
obj = tpipe.o
opt = -g -c
all:    tpipe
tpipe: $(obj)
        gcc     $(obj) -l pthread -o tpipe
tpipe.o: $(src)
        gcc     $(opt) $(src)
clean:
        rm tpipe *.o
```

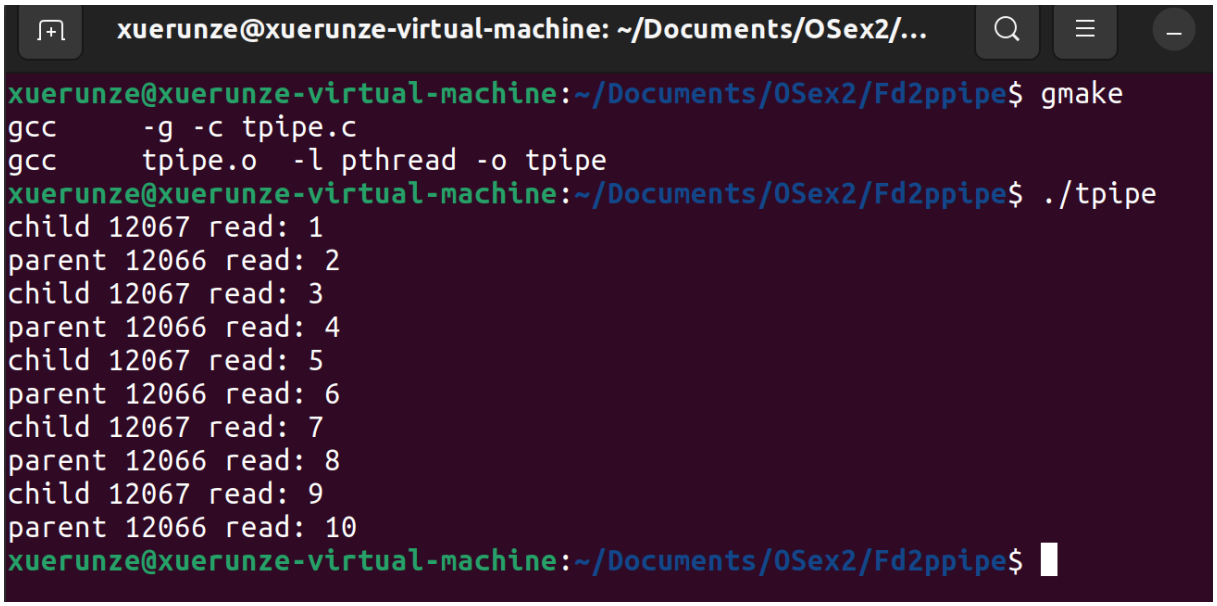
使用 make 命令编译连接生成可执行文件后 tpipe，然后执行./tpipe。





(2)

tpipe.c 的 C 语言程序，makefile 文件等的执行如（1）所述。运行./tpipe，结果如下图：



二、独立实验描述

思路：独立实验要求我们使用 3 个线程来并行协同完成 f(x),f(y)以及 f(x+y)的计算。其中 f(x)是计算 x 的阶乘，f(y)是计算第 y 项斐波那契数列，f(x+y)是计算 f(x)+f(y)。所以可以先在两个线程中计算 f(x)和 f(y)，并建立 pipe1[2]和 pipe2[2]的两个管道。计算结束后将答案写入到两个管道 pipe1[1]和 pipe2[1]中。第三个线程一直在循环读取两个管道 pipe1[0]，pipe2[0]的值，在前两个线程计算完成前，这两个管道中的值都是我们最开始赋给的初始值（我选择将两个管道赋值为 0），当两个管道都

同时不为 0 时，证明前两个线程都计算完成。然后将两个进行加和，并输出加和结果。
线程建立如下图：

```
//使用 pthread_create 系统调用建立3个线程。建立不成功程序退出，执行终止
ret = pthread_create(&thrd1,NULL,(void *) task1,(void *) &num1);
if(ret){
    perror("pthread_create: task1");
    exit(EXIT_FAILURE);
}
ret = pthread_create(&thrd2,NULL,(void *) task2,(void *) &num2);
if(ret){
    perror("pthread_create: task2");
    exit(EXIT_FAILURE);
}
ret = pthread_create(&thrd3,NULL,(void *) task3,NULL);
if(ret){
    perror("pthread_create: task3");
    exit(EXIT_FAILURE);
}
```

以计算 f(x)阶乘为例，计算得到结果后，将答案写入管道 pipe1[1]。管道操作如下图：

```
int zero = 0;
write(pipe1[1],&zero, sizeof(int));
while(cnt){
    x *= cnt;
    cnt--;
    printf("now f(x) = %d\n", x);
}
printf("f(x) final answer is %d\n" ,x);
write(pipe1[1],&x,sizeof(int));
//读写完成后,关闭管道
close(pipe1[1]);
}
```

我选择进行 f(5)，f(6)，以及 f(5+6)的运算。
结果如下：

```
xuerunze@xuerunze-virtual-machine:~/Documents/OSex2/Independent$ ./independent
please write x and y:
5 6
now f(y) = 2
now f(y) = 3
now f(y) = 5
now f(y) = 8
f(y) final answer is8
now f(x) = 5
now f(x) = 20
now f(x) = 60
now f(x) = 120
now f(x) = 120
f(x) final answer is 120
120  8
f(x+y) = 128
xuerunze@xuerunze-virtual-machine:~/Documents/OSex2/Independent$
```

如上图所示，当 f(x)，f(y)的线程计算完成后，通过管道将答案传递到 f(x+y)的线程，f(x+y)收到 f(x),f(y)的值后完成计算，输出结果。

问题回答：

1、 进/线程的协作和通信有哪些特征和功能

进程是独立执行的实体，而线程需要共享资源。线程之间的协作可以通过 pipe 管道来辅助进行，pipe 管道可以帮助线程之间进行通信。Pipe 中数据只能单向流动，而且只能在有亲缘关系的进程中使用。(实验中是在父进程中开辟 pipe，然后不同子进程进行读写)

2、 在真实的操作系统中它是怎样实现和反映出教材中进/线程通信概念的

对于进程：例如 signal 信号，比如定义了键盘中断信号后，使用 kill 从一个进程将信号发送到另一个进程

对于线程：在父进程中开辟 pipe 管道，子线程进行通信。

3、 新理解：管道机制是一种类似队列的数据结构，而进程中的信号机制不是这种结构。

4、 管道机制的机理：管道是在内存中单独构成的一种文件系统，数据在管道中的处理遵循先入先出原则。

5、 怎样利用管道完成进/线程间的协作和通信？

创建管道，在父进程下创建线程和进程（因为匿名管道之间通信需要有亲缘关系），然后在管道中写入和读出数据，pipe[0]是读端，pipe[1]是写端。完成操作后关闭管道。

结论分析：

从样例来看，对于两个线程协作完成加法，两个线程是依次进行的。但是从独立实验来看，虽然建立线程是连续的三条语句，但是执行看起来却不是样例中依次进行的样子。而且先进行了 f(x)线程的创建，反而先进行了 f(y)的计算。分析是由于硬件原因和计算操作原因。

结论： 线程的运行要结合实际操作和硬件情况，不是绝对的每个线程依次执行。